

Livermore
An Evolutionary Rules-Based AI Library
for ANSI Common Lisp

`https:`
`//github.com/cgore/livermore/`

Christopher Mark Gore
`http://www.cgore.com`
`cgore@cgore.com`

Monday, August 17th, AD 2026

Contents

Copyright	ix
Introduction	xi
1 The livermore/learning-parameters Package	1
1.1 Classes	1
1.1.1 The learning-parameters Class	1
2 The livermore/xcs-predicate Package	3
2.1 Classes	3
2.1.1 The xcs-predicate Class	3
2.2 Generics	3
2.2.1 The cover Generic	3
2.2.2 The covering-score Generic	4
2.2.3 The covering? Generic	4
2.2.4 The identical? Generic	4
2.2.5 The match? Generic	4
2.2.6 The more-general? Generic	4
2.2.7 The mutate Generic	5
2.2.8 The duplicate Generic	5
3 The livermore/xcs-ternary-predicate Package	7
3.1 Types	7
3.1.1 The ternary-value Type	7
3.1.2 The ternary-value? Function	7
3.2 Classes	7
3.2.1 The ternary-predicate Class	7
3.2.2 The ternary-predicate Function	7
3.3 Methods	8
4 The livermore/xcs-set-predicate Package	9
4.1 Classes	9
4.1.1 The set-predicate Class	9
4.2 Methods	9
4.2.1 The non-members Function	9

4.2.2	The universal? Function	10
4.2.3	The equivalent? Function	10
5	The livermore/xcs Package	11
5.1	Classes	11
5.1.1	The classifier Class	11
5.1.2	The xcs Class	12
5.1.3	The environment Class	12
5.1.4	The reinforcement-program Class	12
5.1.5	The experiment Class	12
5.2	Match, Action, and Update	13
5.2.1	The generate-match-set Generic	13
5.2.2	The generate-covering-classifier Generic	13
5.2.3	The system-prediction Generic	13
5.2.4	The select-action-learning Generic	13
5.2.5	The generate-action-set Generic	13
5.2.6	The update-set Generic	13
5.2.7	The update-fitness Generic	14
5.3	The Genetic Algorithm	14
5.3.1	The run-ga? Generic	14
5.3.2	The run-ga Generic	14
5.3.3	The select-offspring Generic	14
5.3.4	The crossover Generic	14
5.3.5	The could-subsume? Generic	14
5.3.6	The subsume? Generic	14
5.3.7	The deletion-vote Generic	14
5.3.8	The delete-from-population Generic	15
5.3.9	The insert-into-population Generic	15
5.3.10	The do-action-set-subsumption Generic	15
5.4	The Experiment Loop	15
5.4.1	The start Generic	15
5.4.2	The single-step-experiment Generic	15
5.4.3	The multi-step-experiment Generic	15
5.4.4	The get-situation Generic	15
5.4.5	The get-reward Generic	16
5.4.6	The terminate? Generic	16
5.4.7	The single-step-output Generic	16
5.5	Identity and Generality	16
6	The livermore/xcsr Package	17
6.1	Classes	17
6.1.1	The range-predicate Class	17
6.1.2	The xcsr-learning-parameters Class	17
6.1.3	The xcsr Class	18
6.2	Methods	18

7 The livermore/xcs-analyzer Package	19
7.1 Classes	19
7.1.1 The analyzer Class	19
7.2 Methods	20
7.2.1 The correct-action Generic	20
7.2.2 The correct-action? Generic	20
7.2.3 The get-reward Generic	20
7.2.4 The execute-action Generic	20
8 The livermore/tmscs Package	21
8.1 Classes	21
8.1.1 The tms-predicate Class	21
8.1.2 The tms-classifier Class	22
8.1.3 The tmscs-learning-parameters Class	22
8.1.4 The tmscs Class	22
8.2 Methods	22
9 The livermore/statistics Package	23
9.1 Generics	23
9.1.1 The arithmetic-mean Generic	23
9.1.2 The sample-variance Generic	23
9.1.3 The unbiased-sample-variance Generic	23
9.1.4 The sample-standard-deviation Generic	23
9.1.5 The unbiased-sample-standard-deviation Generic	24
9.1.6 The stat-report Generic	24
10 The livermore/time Package	25
10.1 Decoded Time	25
10.1.1 The decoded-time-list Generic	25
10.1.2 The get-decoded-time-list Function	25
10.1.3 The month-to-number Function	25
10.1.4 The month-name Function	26
10.1.5 The month-short-name Function	26
10.1.6 The encode-month-year Function	26
10.1.7 The encode-year Function	26
10.2 Parsing	26
10.2.1 The yyyy-mm-dd-to-universal-time Function	26
10.2.2 The dd-month-yyyy-to-universal-time Function	26
10.2.3 The date-string-to-universal-time Function	26
10.3 Julian and Ordinal Dates	26
10.3.1 The time-julian-day-number Function	26
10.3.2 The time-julian-date Function	26
10.3.3 The time-ordinal-date Function	27
10.4 Formatting	27

11 The livermore/threshold Package	29
11.0.1 The threshold-indicator Generic	29
12 The livermore/multiplexer Package	31
12.1 Functions	31
12.1.1 The random-bit Function	31
12.1.2 The random-bit-vector Function	31
12.1.3 The binary-decoder Function	31
12.1.4 The multiplexer-length Function	31
12.1.5 The multiplexer Function	32
12.1.6 The truth-vector Function	32
13 The livermore/csv Package	33
13.1 Parameters	33
13.1.1 The *separator* Parameter	33
13.1.2 The *quote* Parameter	33
13.2 Functions	33
13.2.1 The character-list-from-stream Function	33
13.2.2 The parse-character-list Function	33
13.2.3 The parse-stream Function	34
13.2.4 The parse-file Function	34
14 The livermore/stocks Package	35
14.1 Parameters	35
14.1.1 The *initial-money* Parameter	35
14.1.2 The *livermore-data-directory* Parameter	35
14.2 Classes	35
14.2.1 The table-record Class	35
14.2.2 The table Class	36
14.3 Loading	36
14.3.1 The canonical-ticker Function	36
14.3.2 The table-filename Function	36
14.3.3 The load-table Function	36
14.4 Changes and Statistics	37
14.5 Indicators	37
15 The livermore/stock-ticker-descriptions Package	39
15.0.1 The stock-description Generic	39
16 The livermore/stocks-xcs Package	41
16.1 Classes	41
16.1.1 The stocks-xcs-analyzer Class	41
16.1.2 The stocks-experiment Class	41
16.2 Methods	42

17 The livermore/animat-xcs Package	43
17.1 Classes	43
17.1.1 The animat-analyzer Class	43
17.1.2 The animat-experiment Class	43
17.2 Methods	43
17.2.1 The sensor-code Function	44
17.2.2 The start-animat-experiment Function	44
18 The livermore/whitley-test-functions Package	45
18.1 Functions	45
18.1.1 The f1 Function	45
18.1.2 The f2 Function	45
18.1.3 The f3 Function	47
18.1.4 The f4 Function	47
18.1.5 The f5 Function	47
18.1.6 The f6 Function	48
18.1.7 The f7 Function	48
18.1.8 The f8 Function	48
18.1.9 The f9 Function	50
18.1.10 The f10 Function	50
Bibliography	53

Copyright

Copyright © 2005 – 2026, Christopher Mark Gore,

Soli Deo Gloria,

All rights reserved.

22 Forest Glade Court, Saint Charles, Missouri 63304 USA.

Web: <http://www.cgore.com>

Email: cgore@cgore.com

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of Christopher Mark Gore nor the names of other contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS “AS IS” AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Introduction

This library originated from my master's thesis, *A Time Series Classifier* [3], written at the Missouri University of Science and Technology. That work introduced Tsc, a learning classifier system for modeling and predicting time series, derived from Wilson's XCSR [8]. Livermore is that code, grown into a general implementation of XCS [1, 7], XCSR, and the time-multiseries classifier system (TMSCS, the later name for Tsc [3]), with the surrounding problem environments and stock-table utilities.

A time series is a sequence of data measured at successive intervals. Time series analysis is the set of methods used to understand such data, either to explain the underlying system or to predict future points [3]. Weather and the financial markets are the usual historical examples; there are many more real-world time series problems.

No learning classifier system before that thesis had been designed for time series. Existing systems were generally limited to Markov processes with no memory, which we viewed as a major limitation of LCS's [3]. Learning classifier systems exist to evolve an effective rule set for a specified problem, which is specifically the sort of capability that would be desirable for time series analysis and prediction: generating useful rule sets.

The library is named for Jesse Lauriston Livermore, the early twentieth-century speculator whose trading rules were explicit, situational, and revised by experience [5]. That is the same shape as a learning classifier system: a population of rules, each pairing a condition with an action, whose fitness is updated from the payoff they actually earn.

The XCS implementation follows the algorithmic description of Butz and Wilson [1]. Their paper names are kept in the source and in this document: the population is $[P]$, the match set is $[M]$, the action set is $[A]$, and so on. The thesis is available at <https://cgore.com/cgore-thesis-final.pdf>.

You can download the library from GitHub at: <https://github.com/cgore/livermore>
The companion utility library Σ [4] is at: <https://github.com/cgore/sigma>

Getting Lisp

Before using this library you need a working Lisp. I use and recommend SBCL, Steel Bank Common Lisp, which is available at:

<http://www.sbcl.org>

If you are using Debian or a similar Linux distribution (including Ubuntu), you can run as root:

```
apt-get install rlwrap sbcl libpq-dev texlive-full
```

On Fedora or Red Hat:

```
sudo dnf install sbcl rlwrap curl postgresql-devel texlive-scheme-full
```

Getting EMACS and SLIME

After installing, the best way to interact with any Common Lisp is via SLIME, the Superior Lisp Interaction Mode for EMACS, which is available at:

<http://common-lisp.net/project/slime/>

This can be installed on Debian by:

```
apt-get install slime emacs emacs-goodies-el
```

Quicklisp

Livermore depends on `Σ` [4], `md5`, and `clsql`. `Σ` supplies sequences, control, random sampling, probability, time-series types, and the behavior tests used throughout this library. The usual way to obtain the dependencies is Quicklisp:

```
curl -O https://beta.quicklisp.org/quicklisp.lisp
sbcl --load quicklisp.lisp \
      --eval '(quicklisp-quickstart:install)' \
      --eval '(ql:add-to-init-file)' \
      --eval '(quit)'
```

Using the Library

Clone the repository and register it with ASDF. There are several ways to do that; a symlink into a directory already on `asdf:*central-registry*` is enough.

```
mkdir -p ~/programming/lisp
cd ~/programming/lisp
git clone git@github.com:cgore/livermore.git
```

Then, in the REPL:

```
(asdf:load-system :livermore)
```

A typical single-step experiment builds an environment, an XCS, and an experiment, then calls start:

```
(let* ((lp (make-instance 'livermore/learning-parameters:learning-parameters
                          :minimum-number-of-actions 2
                          :possible-actions '(0 1)))
      (xcs (make-instance 'livermore/xcs:xcs
                          :learning-parameters lp)))
  xcs)
```

The chapters that follow document each package. Start with `livermore/learning-parameters` and `livermore/xcs` if you want the classifier system itself; start with `livermore/stocks` if you want the market data.

Building this Document

From the documentation directory:

```
make
```

That produces `livermore.dvi`, `livermore.ps`, and `livermore.pdf`.

Chapter 1

The livermore/learning-parameters Package

The `livermore/learning-parameters` package holds the numeric knobs for an XCS run. Each slot is an accessor on the `learning-parameters` class. The names in quotes below are those used by Butz and Wilson [1]; the defaults match the values they recommend.

1.1 Classes

1.1.1 The `learning-parameters` Class

An instance of `learning-parameters` is passed to every XCS. Only `minimum-number-of-actions` has no default; it must be supplied, and is usually equal to the number of legal actions so that covering can fill any missing action.

Syntax

```
(make-instance 'learning-parameters :minimum-number-of-actions  
n &key ...)
```

Arguments and Values

maximum-total-numerosity N in the paper, usually in the hundreds.
Default 100.

learning-rate β , usually in $[0.1, 0.2]$. Default 0.1.

multiplier-parameter α , usually 0.1. Default 0.1.

equal-error-threshold ϵ_0 , usually about one percent of the reward.
Default 10.0.

power-parameter ν , usually 5. Default 5.

discount-factor γ , usually 0.71. Default 0.71.

ga-threshold θ_{GA} , usually in $[25, 50]$. Default 50.

crossover-probability χ , usually in $[0.5, 1.0]$. Default 0.5.

mutation-probability μ , usually in $[0.01, 0.05]$. Default 0.025.

deletion-threshold θ_{del} , usually 20. Default 20.

fitness-fraction-threshold Used by deletion-vote. Default 0.1.

minimum-subsumption-experience θ_{sub} , usually at least 20. Default 20.

covering-probability $P_{\#}$, the chance that a covering allele is don't-care. Default 0.33.

initial-prediction p_I , slightly more than zero. Default 0.001.

initial-prediction-error ϵ_I . Default 0.001.

initial-fitness F_I . Default 0.001.

exploration-probability P_{explr} . Default 0.5.

minimum-number-of-actions θ_{mna} . Required.

maximum-number-of-steps The cap on a multi-step trial. Default 50.

ga-subsumption? `doGASubsumption`. Default true.

action-set-subsumption? `doActionSetSubsumption`. Default true.

possible-actions The legal actions. Default `(nil t)`.

Examples

```
(make-instance 'learning-parameters
  :minimum-number-of-actions 2
  :possible-actions '(0 1)
  :maximum-total-numerosity 800)
```


Chapter 2

The livermore/xcs-predicate Package

The `livermore/xcs-predicate` package is the protocol for a single allele of an XCS condition. Ternary bits, sets, intervals, and time-series paths are all subclasses of `xcs-predicate`. Specialize the generics below for a new representation.

2.1 Classes

2.1.1 The `xcs-predicate` Class

The empty base class. Make a subclass for each representation.

2.2 Generics

2.2.1 The `cover` Generic

Generate a covering predicate that matches *situation*. This is the operation used by `generate-covering-classifier`.

Syntax

```
(cover predicate situation learning-parameters)
```

A method on `symbol` builds an instance of that class and covers with it, so `(cover 'ternary-predicate situation lp)` is legal.

2.2.2 The covering-score Generic

A numeric score for how general the predicate is. Used when choosing subsumers.

Syntax

```
(covering-score predicate learning-parameters)
```

The default method returns 0.

2.2.3 The covering? Generic

True if the predicate matches every possible situation.

Syntax

```
(covering? predicate)
```

The default method returns false.

2.2.4 The identical? Generic

True if *a* and *b* are the same for all situations.

Syntax

```
(identical? a b)
```

2.2.5 The match? Generic

True if *predicate* matches *situation*. The default method signals an error; subclasses must specialize it.

Syntax

```
(match? predicate situation)
```

2.2.6 The more-general? Generic

True if *general* is strictly more general than *specific*.

Syntax

```
(more-general? general specific)
```

2.2.7 The mutate Generic

Mutate *predicate* according to *learning-parameters*, preserving a match on *situation*. The default method signals an error.

Syntax

```
(mutate predicate situation learning-parameters)
```

2.2.8 The duplicate Generic

Return a newly created copy. The default method copies a `value` slot if one exists.

Chapter 3

The livermore/xcs-ternary-predicate Package

The `livermore/xcs-ternary-predicate` package is the classical XCS allele: `true`, `false`, or `don't-care`. A condition is a sequence of these predicates, one per bit of the situation.

3.1 Types

3.1.1 The ternary-value Type

The type of a ternary allele: `t`, `nil`, or `:#`.

3.1.2 The ternary-value? Function

True when its argument is one of those three values.

3.2 Classes

3.2.1 The ternary-predicate Class

A predicate whose `value slot` is a `ternary-value`. The `don't-care` value `:#` matches both `t` and `nil`.

3.2.2 The ternary-predicate Function

A constructor.

Syntax

```
(ternary-predicate value)
```

Examples

```
(ternary-predicate t)
(ternary-predicate :#)
```

3.3 Methods

`covering?` is true only for `:#`. `covering-score` is 1 or 0 accordingly.

`match?` against a situation bit is true if the predicate is `:#` or if the two boolean senses agree. `match?` against another predicate is true if this predicate matches every situation the other matches.

`more-general?` is true only when the general predicate is `:#` and the specific one is not.

`cover` maps over a situation sequence and, with probability $P_{\#}$, writes `:#` instead of the situation bit.

`mutate` flips between `:#` and the current situation bit with probability μ .

Chapter 4

The livermore/xcs-set-predicate Package

The `livermore/xcs-set-predicate` package is an allele whose match is membership in a set, or the whole universe if the covering flag is set. Share the `possible-members` list among predicates of the same problem.

4.1 Classes

4.1.1 The `set-predicate` Class

members The values matched when the predicate is not covering.

possible-members The universe of values this allele may name.

covering? True if the predicate matches every possible member.

4.2 Methods

4.2.1 The `non-members` Function

The possible members that are not in `members`. Empty when the predicate is covering.

4.2.2 The universal? Function

True if the predicate covers the whole universe, either by the covering flag or by listing every possible member.

4.2.3 The equivalent? Function

True if two predicates match the same situations. A covering predicate and a predicate that lists every possible member are equivalent.

`identical?` is stricter: the covering flags must agree as well.

`match?` against a situation is true if the predicate is covering or the situation is a member. `match?` against another set predicate is true if this predicate matches everything the other matches.

`more-general?` is true when this predicate matches the other and they are not equivalent.

`cover` builds a predicate whose members are the given situations, and sets the covering flag with the given probability.

`mutate` may drop the covering flag, raise it, add a member, or remove a member other than the current situation.

`covering-score` is 1 if covering, otherwise the fraction of possible members listed.

Chapter 5

The livermore/xcs Package

The `livermore/xcs` package is Wilson's XCS [7], written from the algorithmic description of Butz and Wilson [1]. A classifier is a rule $C \rightarrow A$ with a prediction, an error, and a fitness. The system matches the current situation, picks an action, updates the classifiers that advocated that action, and from time to time runs a genetic algorithm on the action set rather than on the whole population.

5.1 Classes

5.1.1 The classifier Class

A single rule. The slots are those of Butz and Wilson:

environment-condition C , a sequence of predicates.

action A , the recommended action.

prediction p , the predicted payoff.

prediction-error ϵ .

fitness F , used by the genetic algorithm.

experience exp .

time-stamp ts .

action-set-size as .

numerosity n , how many identical copies this macroclassifier stands for.

accuracy κ .

5.1.2 The xcs Class

The running system. Important slots:

learning-parameters The parameter set.

population $[P]$.

match-set $[M]$.

action-set $[A]$.

previous-action-set $[A]_{-1}$.

prediction-array $[PA]$.

reward ρ .

previous-reward ρ_{-1} .

payoff P .

predicate-type The class of alleles, usually `ternary-predicate`.

classifier-type Usually `classifier`.

situation σ .

previous-situation σ_{-1} .

action `act`.

number-of-situations How many situations have been seen.

explore? True while the current trial is exploring.

5.1.3 The environment Class

Presents situations and accepts actions. Specialize `get-situation`, `execute-action`, and `reset`. The `multistep?` slot is true when a problem lasts for more than one action.

5.1.4 The reinforcement-program Class

Scores actions. Specialize `get-reward` and `end-of-problem?`.

5.1.5 The experiment Class

Holds an environment, a reinforcement program, and an XCS. Many problems use the same object for the first two.

number-of-trials How many multi-step trials to run. Default 10000.

5.2 Match, Action, and Update

5.2.1 The generate-match-set Generic

The “GENERATE MATCH SET” function. $[M]$ is every classifier in $[P]$ that matches the current situation. Covering classifiers are added until θ_{mna} distinct actions are present.

5.2.2 The generate-covering-classifier Generic

The “GENERATE COVERING CLASSIFIER” function. Builds a classifier whose condition covers the situation and whose action is a random action missing from $[M]$.

5.2.3 The system-prediction Generic

The fitness-weighted mean prediction of classifiers in $[M]$ that advocate *action*.

Syntax

```
(system-prediction xcs action)
```

5.2.4 The select-action-learning Generic

The “SELECT ACTION” function. With probability P_{explr} picks a random action from $[M]$; otherwise picks the action of highest system prediction.

`select-action-random` and `select-action-best` are the two halves of that choice.

5.2.5 The generate-action-set Generic

The “GENERATE ACTION SET” function. $[A]$ is the classifiers in $[M]$ whose action is the chosen one. The previous action set is saved first.

5.2.6 The update-set Generic

The “UPDATE SET” function. Updates prediction, error, action-set size, and fitness for the classifiers in an action set, then may run action-set subsumption.

5.2.7 The update-fitness Generic

The “UPDATE FITNESS” function. Accuracy is 1 when the error is below ϵ_0 , otherwise $\alpha(\epsilon/\epsilon_0)^{-\nu}$. Fitness then moves toward the classifier’s share of the accuracy sum.

5.3 The Genetic Algorithm

5.3.1 The run-ga? Generic

True when the action set is non-empty and the average time since the last GA exceeds θ_{GA} .

5.3.2 The run-ga Generic

The “RUN GA” function. Selects two parents by fitness, crosses them, mutates the children, and either subsumes each child into a parent or inserts it into $[P]$. Insertion may delete a low-fitness classifier if numerosity has reached N .

5.3.3 The select-offspring Generic

The “SELECT OFFSPRING” function. Roulette selection on fitness.

5.3.4 The crossover Generic

The “APPLY CROSSOVER” function. Two-point crossover on the condition, then the midpoint of prediction, error, and fitness.

5.3.5 The could-subsume? Generic

The “COULD SUBSUME” function. True when experience exceeds θ_{sub} and error is below ϵ_0 .

5.3.6 The subsume? Generic

The “DOES SUBSUME” function. True when the actions agree, the general classifier could subsume, and it is more general.

5.3.7 The deletion-vote Generic

The “DELETION VOTE” function. Ranks a classifier for deletion, raising the vote when experience is high and fitness is a small fraction of the average.

5.3.8 The delete-from-population Generic

The “DELETE FROM POPULATION” function. When total numerosity is at least N , reduces the numerosity of a classifier chosen by deletion vote, or removes it if numerosity is 1.

5.3.9 The insert-into-population Generic

The “INSERT IN POPULATION” function. If an identical classifier is already present, its numerosity is increased; otherwise the new classifier is pushed.

5.3.10 The do-action-set-subsumption Generic

The “DO ACTION SET SUBSUMPTION” function. A random most-general subsumer absorbs every more-specific classifier in the action set.

5.4 The Experiment Loop**5.4.1 The start Generic**

Starts an experiment. Multi-step environments use `multi-step-experiment`; single-step environments use `single-step-experiment`.

5.4.2 The single-step-experiment Generic

Runs trials until `terminate?` is true. Each trial explores with probability P_{explr} .

`single-step-explore` matches, acts, updates, and may run the GA. `single-step-exploit` takes the best action and does not learn.

5.4.3 The multi-step-experiment Generic

Runs `number-of-trials` problems. Even-numbered trials (from zero) explore and learn; odd-numbered trials exploit.

`multi-step-explore` uses a discounted payoff $\rho_{-1} + \gamma \max[\text{PA}]$ to update the previous action set. `multi-step-exploit` always takes the best action.

5.4.4 The get-situation Generic

On an experiment, records the previous situation, reads a new one from the environment, and advances the situation counter.

5.4.5 The `get-reward` Generic

On an experiment, records the previous reward and reads a new one from the reinforcement program.

5.4.6 The `terminate?` Generic

True when the experiment should stop. Specialize this on each experiment class.

5.4.7 The `single-step-output` Generic

Optional report after a single-step trial.

5.5 Identity and Generality

`identical?` on two classifiers compares conditions and actions. `match?` on a classifier and a situation tests the condition. `more-general?` on two classifiers tests the conditions. `actions-in` returns the distinct actions in a sequence of classifiers. `duplicate` copies a classifier, including its condition.

Chapter 6

The `livermore/xcsr` Package

The `livermore/xcsr` package is XCS with interval predicates [8]. Each allele is a closed interval $[lower, upper]$ over a real input.

6.1 Classes

6.1.1 The `range-predicate` Class

lower The inclusive lower bound.

upper The inclusive upper bound.

spread is half the width. **center** is the midpoint. If **lower** exceeds **upper**, the two are swapped first.

6.1.2 The `xcsr-learning-parameters` Class

A subclass of `learning-parameters` with extra slots:

problem-range The range of the input, default `(0.0 1.0)`.

covering-maximum How large a fraction of the range covering may add to both bounds. Default 0.1.

mutation-maximum How large a fraction of the range mutation may add or subtract from each bound. Default 0.10.

initial-spread-limit s_0 in Wilson's XCSR paper, the largest initial spread when covering. Default 0.5.

6.1.3 The `xcsr` Class

An XCS whose default predicate type is `range-predicate` and whose learning parameters are `xcsr-learning-parameters`.

6.2 Methods

`match?` against a number is true when the number lies in the interval.

`match?` against another range is true when this interval contains the other.

`more-general?` is true when this interval strictly contains the other.

`cover` builds, for each situation element, an interval centered on that element with a random spread no larger than `initial-spread-limit`.

`mutate` adds a random amount, bounded by `mutation-maximum` times the spread, to each bound, clipped to the problem range.

`covering-score` is the width of the interval as a fraction of the problem range.

Chapter 7

The livermore/xcs-analyzer Package

The `livermore/xcs-analyzer` package is a combined environment and reinforcement program that scores actions and keeps a running history of correctness. Most single-step problems subclass `analyzer` and define `correct-action` and `get-situation`.

7.1 Classes

7.1.1 The analyzer Class

Subclass of both `environment` and `reinforcement-program`.

current-situation The situation most recently presented.

number-of-situations How many situations have been presented.

current-action The action most recently taken.

actions How many actions have been taken.

correct-actions How many of those were correct.

action-history `t` or `nil` for each action, newest first.

odd-action-history The same history for odd-numbered actions.

even-action-history The same history for even-numbered actions.

7.2 Methods

7.2.1 The correct-action Generic

The action that should have been taken. The default method signals an error; define this for each problem.

7.2.2 The correct-action? Generic

True if `current-action` equals `correct-action`.

7.2.3 The get-reward Generic

Returns 1000.0 for a correct action and 0.0 otherwise.

7.2.4 The execute-action Generic

Records the action, updates the correctness counts, and prints a short running report, including the last fifty actions split into odd and even trials.

Chapter 8

The livermore/tmscs Package

The `livermore/tmscs` package is the time series classifier (TSC) of [3], later named TMSCS when the representation was allowed more than one series. It is XCSR whose alleles are paths through a time series rather than intervals on a single real. Situations are time series or time multiseries as defined in Σ [4].

A rule is a collection of predicates; each predicate must match the current situation for the rule to match [3]. A single predicate consists of an initial and a final position, a field selector ϕ , an operator ω , and a range $[l, u]$. Taking the data along the straight line segment from the initial point to the final point forms a vector d ; applying ϕ to each element yields d' . The predicate matches if and only if $l \leq \omega(d') \leq u$. When all of the predicates of the rule match, the rule recommends a particular classification or action.

8.1 Classes

8.1.1 The `tms-predicate` Class

A subclass of `range-predicate`.

initial The start of the path: temporal index first, then array coordinates.

final The end of the path, in the same form.

field A function applied to each cell, default `identity`.

operation A function applied to the sequence of field values, default `identity`.

`tms-predicate` is also a constructor that applies `make-instance`.

8.1.2 The `tms-classifier` Class

A classifier whose condition is a list of `tms-predicates`. It matches a situation when every predicate matches.

8.1.3 The `tmscs-learning-parameters` Class

A subclass of `xcsr-learning-parameters`.

maximum-environment-condition-length How many predicates a covering classifier may have. Default 1.

maximum-temporal-mutation The largest random change to the temporal coordinate. Default 5.

maximum-position-mutation The largest random change to any spatial coordinate. Default 5.

valid-operations The operations covering and mutation may choose.

valid-fields The fields they may choose. Default `(list #'identity)`.

visible-time-range The window of history the classifiers may see. Default `(0 100)`.

8.1.4 The `tmscs` Class

An XCSR whose predicate type is `tms-predicate` and whose classifier type is `tms-classifier`.

8.2 Methods

`path` is the raster line from `initial` to `final`. `field-path` is the field applied to each cell on that path. `operate` applies the operation to that sequence.

`match?` is true when the operated value lies in the range. `more-general?` is true when the ranges and paths nest strictly.

`cover` picks random endpoints in the visible window, a random field and operation, and a range around the operated value.

`mutate` may slide the path a little along its line and move the bounds, keeping the current situation classified as before.

`crossover` is one-point on the list of predicates, and may also swap a prefix of slots inside the predicate at the cut.

`covering-score` is the path length times the range width as a fraction of the problem range.

Chapter 9

The livermore/statistics Package

The `livermore/statistics` package is a small set of sample statistics on sequences. Each generic takes `&key` `key` `start` `end`. The underlying sequence helpers come from Σ [4].

9.1 Generics

9.1.1 The arithmetic-mean Generic

The sample mean $\bar{x}$. Returns `nil` on an empty sequence.

9.1.2 The sample-variance Generic

The second central moment s_N^2 , divided by N .

9.1.3 The unbiased-sample-variance Generic

The unbiased estimator s_{N-1}^2 , divided by $N - 1$. Returns `nil` when the sequence has fewer than two elements.

9.1.4 The sample-standard-deviation Generic

$\sqrt{s_N^2}$.

9.1.5 The unbiased-sample-standard-deviation Generic

$$\sqrt{s_{N-1}^2}.$$

9.1.6 The stat-report Generic

Prints the mean, unbiased standard deviation, minimum, and maximum of *data* to *destination*.

Syntax

```
(stat-report destination data &key key start end pre-string  
format-string post-string)
```

Examples

```
(arithmetic-mean '(1 2 3))      ; => 2  
(unbiased-sample-variance '(1 2 3)) ; => 1
```

Chapter 10

The `livermore/time` Package

The `livermore/time` package names the fields of a Common Lisp decoded time, parses and formats dates, and computes Julian and ordinal dates. Most functions take an optional *time*, which may be a universal time or a decoded-time list. The default is the current time.

10.1 Decoded Time

10.1.1 The `decoded-time-list` Generic

Returns *time* as the nine-value list produced by `decode-universal-time`. An integer is decoded; a list is returned unchanged.

10.1.2 The `get-decoded-time-list` Function

The current time as a decoded-time list.

The accessors `time-second`, `time-minute`, `time-hour`, `time-date`, `time-month`, `time-year`, `time-day`, `time-daylight-p`, and `time-zone` are the nine fields in order. `time-day` is the weekday, Monday = 0 through Sunday = 6. `time-month-name` and `time-month-short-name` are English names. `time-quarter` is 1 through 4.

10.1.3 The `month-to-number` Function

The month number for an English name, January = 1. Short names such as "Jan" are accepted.

10.1.4 The month-name Function

The full English name for a month number.

10.1.5 The month-short-name Function

The abbreviated English name.

10.1.6 The encode-month-year Function

The universal time of the first of *month* in *year*.

10.1.7 The encode-year Function

The universal time of 1 January of *year*.

10.2 Parsing

10.2.1 The yyyy-mm-dd-to-universal-time Function

Parses a string of the form YYYY-MM-DD.

10.2.2 The dd-month-yyyy-to-universal-time Function

Parses a string of the form DD-Mon-YYYY or DD-Monthname-YY.

10.2.3 The date-string-to-universal-time Function

Chooses between those two by the length of the middle field.

`dd-month-yyyy-to-decoded-time` and `dd-month-yyyy-to-decoded-time-list` decode the same strings.

10.3 Julian and Ordinal Dates

10.3.1 The time-julian-day-number Function

The Julian Day Number at noon for a Gregorian date. Years work as 1 BC = 0, 2 BC = -1, and so on. Also called `time-jdn`.

10.3.2 The time-julian-date Function

The Julian Date, including the fraction of the day past noon. Also called `time-jd`.

10.3.3 The time-ordinal-date Function

The day of the year, 1 through 366. Also called `time-day-of-year`.

10.4 Formatting

yyyy-string "2020"

qqyyyy-string "1Q2020"

qq-yyyy-string "Q1-2020"

mon-yyyy-string "Mar-2020"

dd-mon-yyyy-string "15-Mar-2020"

yyyy-mm-dd-string "2020-3-15", also `iso-extended-date-string`
and `iso-date-string`

yyyymmdd-string unpadded YYYYMMDD, also `iso-short-date-string`

date-time-string month, weekday (Monday = 0), year, and zero-
padded hours, minutes, and seconds

Chapter 11

The `livermore/threshold` Package

The `livermore/threshold` package is a single generic used as a target function for XCSR: whether a point meets or exceeds a threshold.

11.0.1 The threshold-indicator Generic

True if *instance* meets or exceeds *threshold*.

A number meets a number when $instance \geq threshold$. A list or vector meets another of the same length when every coordinate satisfies that comparison.

Syntax

```
(threshold-indicator threshold instance)
```

Examples

```
(threshold-indicator 5 6)           ; => T
(threshold-indicator '(1 2) '(2 2)) ; => T
(threshold-indicator #(1 2) #(2 1)) ; => NIL
```


Chapter 12

The livermore/multiplexer Package

The `livermore/multiplexer` package is the Boolean multiplexer used as Wilson's classic single-step XCS problem [7]. An `address-width` of k takes a bit vector of length $k + 2^k$: k address bits and 2^k data bits. The output is the data bit named by the address.

12.1 Functions

12.1.1 The `random-bit` Function

Returns 0 or 1.

12.1.2 The `random-bit-vector` Function

A random bit vector of the given positive length.

12.1.3 The `binary-decoder` Function

Decodes a sequence of bits as an unsigned integer. `nil` or 0 is a zero bit; anything else is a one bit.

12.1.4 The `multiplexer-length` Function

The bit-vector length for a given address width: $k + 2^k$.

12.1.5 The multiplexer Function

The multiplexer output.

Syntax

```
(multiplexer address-width bits)
```

Examples

```
(multiplexer 2 #*110001) ; address 3 selects the last data bit => 1  
(multiplexer 1 #*010)    ; address 0 selects the first data bit => 1
```

12.1.6 The truth-vector Function

A general vector of `t` and `nil` corresponding to the 1 and 0 bits of a bit vector.

Chapter 13

The `livermore/csv` Package

The `livermore/csv` package reads comma-separated files into lists of lists. Numbers are parsed with `read-from-string`; everything else is left as a string. The first row is not treated as a header.

13.1 Parameters

13.1.1 The `*separator*` Parameter

The column separator. Default comma.

13.1.2 The `*quote*` Parameter

The string delimiter. Default the double-quote character. A quote is closed by a second quote; escaped quotes inside a field are not supported.

13.2 Functions

13.2.1 The `character-list-from-stream` Function

Reads every character from *stream* until the stream ends.

13.2.2 The `parse-character-list` Function

The parser. Returns a list of rows, each a list of fields.

13.2.3 The `parse-stream` Function

Reads a stream and parses it.

13.2.4 The `parse-file` Function

Opens *pathname* and parses it.

Syntax

(`parse-file` *pathname*)

Chapter 14

The `livermore/stocks` Package

The `livermore/stocks` package is a price table: daily bars loaded from Yahoo-style CSV files, optional monthly and yearly aggregates, and a set of moving-average and directional signals. The working series is (`records table`), which applies `preferred-records` and then clips to `records-start` and `records-end`.

14.1 Parameters

14.1.1 The `*initial-money*` Parameter

The default cash stake for the analyzers. Default 100000.00.

14.1.2 The `*livermore-data-directory*` Parameter

The directory of Yahoo CSV files, under the ASDF system source directory.

14.2 Classes

14.2.1 The `table-record` Class

One OHLC bar.

`opening-time`, `closing-time` Universal times.

`descriptive-time` A printable date.

opening-price, high-price, low-price, closing-price Unadjusted prices.

trading-volume Session volume.

adjusted-closing-price Split-adjusted close.

adjustment is adjusted close over close. **adjusted-opening-price**, **adjusted-high-price**, and **adjusted-low-price** multiply the unadjusted prices by that factor. **table-record** is also a constructor.

14.2.2 The table Class

A price series for one ticker.

ticker-symbol Canonical ticker.

preferred-records A function of the table that returns the working bars, default #'daily-records.

records-start, records-end The slice of that series.

daily-records is the raw daily series. **monthly-records** and **yearly-records** build OHLC aggregates on first use.

On a table, **opening-time**, **opening-price**, and the other price accessors map over records and return a list.

14.3 Loading

14.3.1 The canonical-ticker Function

Uppercase ticker string, following Yahoo's style.

14.3.2 The table-filename Function

The path of the local *TICKER*-table.csv.

14.3.3 The load-table Function

Also called **load-table-from-yahoo**. Reads that file, skips the header row, and sorts the bars by opening time.

Syntax

```
(load-table ticker-symbol &key preferred-records)
  retrieve-stock-data fetches a Yahoo table.csv with wget. retrieve-stocks-data
  does so for every ticker in *ticker-descriptions*.
```

14.4 Changes and Statistics

difference, value-ratio, and percent-change compare a key at start and end. percent-change is 100 times the value ratio. spread-difference and its siblings compare position+spread to position.

The sample statistics from `livermore/statistics` are specialized on table. `sample-z-score` and `unbiased-sample-z-score` are the *z-score* of the key at start.

`shares-buyable` is how many whole shares a sum of money buys at the adjusted close. `buy-and-hold` grows an initial sum by the value ratio.

14.5 Indicators

`simple-moving-average` (`sma`) is the unweighted mean of each window. `arithmetically-weighted-moving-average` (`awma`) weights the newest point most. `exponential-moving-average` (`ema`) uses $\alpha = 2/(n+1)$.

`moving-average-convergence-divergence` (`macd`) is the difference of two EMAs. `macd-signal` is an EMA of that difference.

`upward-changes` and `downward-changes` are the positive day-to-day moves, else 0. `relative-strength` is the ratio of a moving average of those two series. `ema-rs` is Wilder's form; `ema-rs-27` is the textbook period.

`simple-directional-signal` is true where the price *period* steps later is higher. The matching `*-gains` and `*-performance` functions turn a signal into a product of day-to-day returns, holding cash when the signal is false.

`sma-crossover-signal` is true when the short SMA exceeds *ratio* times the long SMA. `directional-sma-crossover-signal` returns `:long`, `:short`, or `:cash` from those averages and the slope of the long average. The `*-optimal` functions search period and ratio.

Chapter 15

The livermore/stock-ticker-description Package

The `livermore/stock-ticker-descriptions` package is a table of ticker symbols and long names. The lists `*dow-jones-indices*`, `*nyse-indices*`, `*nasdaq-indices*`, `*standard-and-poors-indices*`, `*other-us-indices*`, `*us-treasury-indices*`, `*us-commodities-indices*`, `*misc-stocks*`, and `*^dji-components*` are `alist`s of (ticker description). `*ticker-descriptions*` is a hash table of the same data.

15.0.1 The stock-description Generic

The long name of a ticker, or `nil` if it is unknown. Methods exist for strings and symbols; `livermore/stocks` adds a method on `table`.

Chapter 16

The livermore/stocks-xcs Package

The `livermore/stocks-xcs` package walks a price table and asks XCS whether to be in the stock or in cash. The situation is a vector of extrema and z -score features from recent prices and volume. Reward is chosen by `*reward-method*`.

16.1 Classes

16.1.1 The `stocks-xcs-analyzer` Class

A subclass of `analyzer`.

table The price table.

initial-index, current-index The bar under consideration.

money, previous-money Cash marked to market.

previous-action The last action.

16.1.2 The `stocks-experiment` Class

An experiment that uses a `stocks-xcs-analyzer`. `terminate?` is true at the last record or when money is not positive.

16.2 Methods

`current-record` and `previous-record` are the bars at and before `current-index`.

`should-have-bought-stock?` is true when today's adjusted close is above yesterday's. `buy-stock?` is true when the current action is `:stock`, or when it is `:hold` and the previous action was `:stock`. `correct-action?` is true when that buy decision matches whether the price rose.

`money-from-stock` marks to today's close as many shares as yesterday's cash would buy.

`execute-action` advances one bar and, if buying, updates money.

`get-reward` returns a value chosen by `*reward-method*` (`:money`, `:delta-money`, `:weighted-delta-money`, `:diff-money`, `:weighted-diff-money`, or `:correctness`) and may print a periodic report.

`load-*table*` loads a ticker into `*table*`.

Chapter 17

The livermore/animat-xcs Package

The `livermore/animat-xcs` package is Woods-style maze navigation, the classic multi-step XCS problem. The animat sees the eight neighboring cells and must walk to food. Maze files live under `data/animat-data/`.

17.1 Classes

17.1.1 The `animat-analyzer` Class

An environment and reinforcement program. Construction reads `file-name:` width, height, then one character per cell. `*` is empty, `O` and `Q` are rocks, `F` and `G` are food.

The situation is a 24-element truth list: a three-bit sensor code for each of the eight neighbors, counter-clockwise from north. Edges wrap.

17.1.2 The `animat-experiment` Class

An experiment that uses an `animat-analyzer`.

17.2 Methods

`reset` places the animat on a random empty cell. `execute-action` moves in one of eight directions; a rock is a no-op. `get-reward` is 1000 on food and 0 otherwise. `end-of-problem?` is true on food.

17.2.1 The sensor-code Function

The three-bit code for a cell symbol.

17.2.2 The start-animat-experiment Function

Builds an experiment from the maze in *file-name* and starts it.

Syntax

```
(start-animat-experiment file-name)
```

Examples

```
(start-animat-experiment  
  (asdf:system-relative-pathname "livermore"  
    "data/animat-data/Woods1.txt"))
```

Chapter 18

The livermore/whitley-test-functions Package

The `livermore/whitley-test-functions` package is the test suite of Whitley, Mathias, Rana, and Dzubera [6], including De Jong's classical functions [2]. Each `fn` takes a list of floats in the published domain. The figures below are two-variable slices (remaining coordinates, if any, are zero), drawn with PGFPlots. Darker blue is lower; yellow is higher.

18.1 Functions

18.1.1 The `f1` Function

F_1 is De Jong's sphere, a unimodal sum of squares in three variables.

$$f_1(\mathbf{x}) = \sum_{i=1}^3 x_i^2, \quad x_i \in [-5.12, 5.11]. \quad (18.1)$$

The global minimum is 0 at the origin.

18.1.2 The `f2` Function

F_2 is De Jong's two-variable Rosenbrock function.

$$f_2(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2, \quad x_i \in [-2.048, 2.047]. \quad (18.2)$$

The global minimum is 0 at (1,1), at the end of a narrow curved valley.

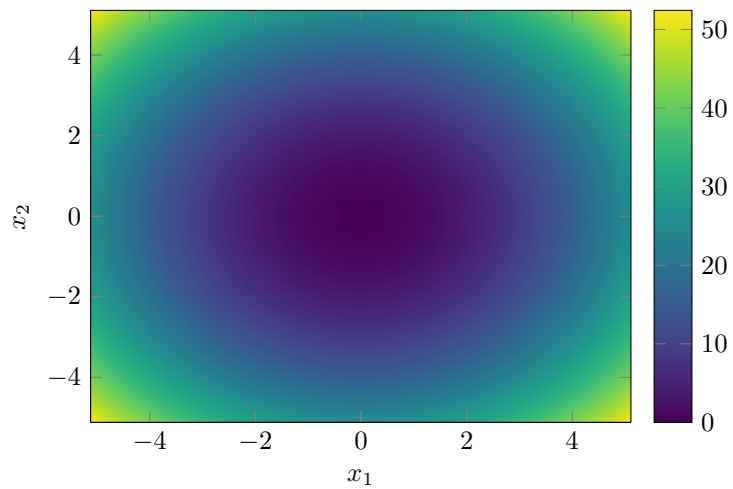


Figure 18.1: f_1 on the x_1x_2 plane with $x_3 = 0$.

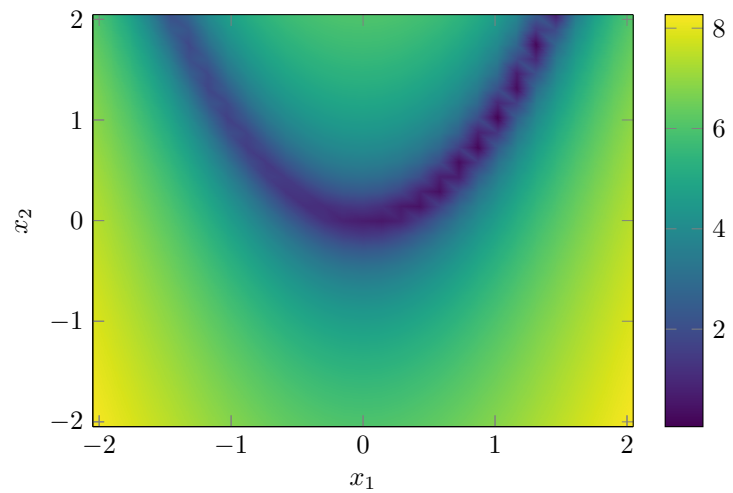


Figure 18.2: f_2 , shown on a $\log(1 + f)$ scale so the valley is visible.

18.1.3 The f_3 Function

F_3 is De Jong's step function, a sum of floors in five variables.

$$f_3(\mathbf{x}) = \sum_{i=1}^5 \lfloor x_i \rfloor, \quad x_i \in [-5.12, 5.11]. \quad (18.3)$$

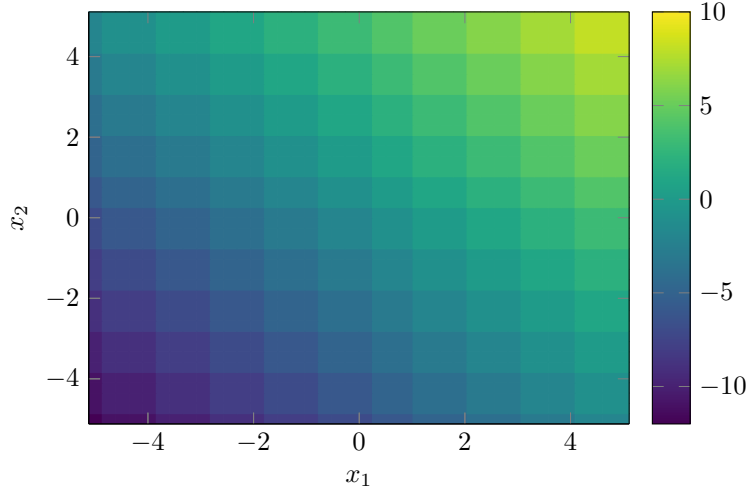


Figure 18.3: f_3 on the x_1x_2 plane with the other coordinates zero.

18.1.4 The f_4 Function

F_4 is De Jong's noisy quartic in thirty variables. $N(0, 1)$ is a standard normal sample, drawn anew on each call.

$$f_4(\mathbf{x}) = \sum_{i=1}^{30} i x_i^4 + N(0, 1), \quad x_i \in [-1.28, 1.27]. \quad (18.4)$$

18.1.5 The f_5 Function

F_5 is Shekel's foxholes, a two-variable function with twenty-five local wells.

$$f_5(\mathbf{x}) = \left(0.002 + \sum_{j=1}^{25} \frac{1}{j + \sum_{i=1}^2 (x_i - a_{ij})^6} \right)^{-1}, \quad x_i \in [-65.536, 65.535]. \quad (18.5)$$

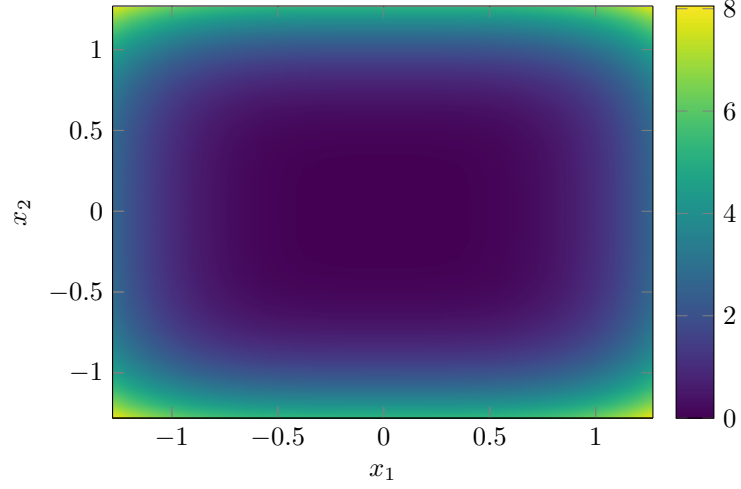


Figure 18.4: The deterministic part of f_4 in two variables, $x_1^4 + 2x_2^4$, with the remaining coordinates zero and no noise.

The columns of $A = (a_{ij})$ are the grid $\{-32, -16, 0, 16, 32\} \times \{-32, -16, 0, 16, 32\}$, in that order. Each foxhole is a local minimum near $1/(j + 0.002)$.

18.1.6 The f6 Function

F_6 is the Rastrigin function, also called `rastrigin`. The dimension n is the length of the argument.

$$f_6(\mathbf{x}) = 10n + \sum_{i=1}^n (x_i^2 - 10 \cos(2\pi x_i)), \quad x_i \in [-5.12, 5.11]. \quad (18.6)$$

The global minimum is 0 at the origin.

18.1.7 The f7 Function

F_7 is the Schwefel function, also called `schwefel`.

$$f_7(\mathbf{x}) = \sum_{i=1}^n -x_i \sin \sqrt{|x_i|}, \quad x_i \in [-512, 511]. \quad (18.7)$$

This is the raw Schwefel sum, without the usual $418.9829n$ offset.

18.1.8 The f8 Function

F_8 is the Griewangk function, also called `griewangk`.

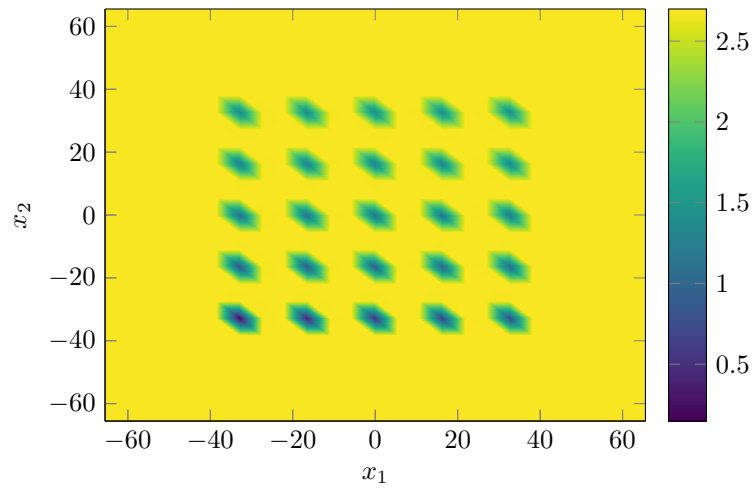


Figure 18.5: f_5 on a $\log_{10}$ scale. The twenty-five dark wells are the foxholes.

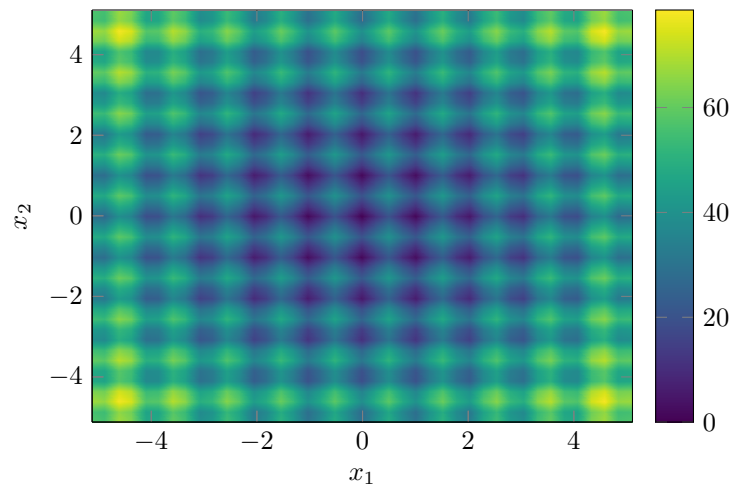
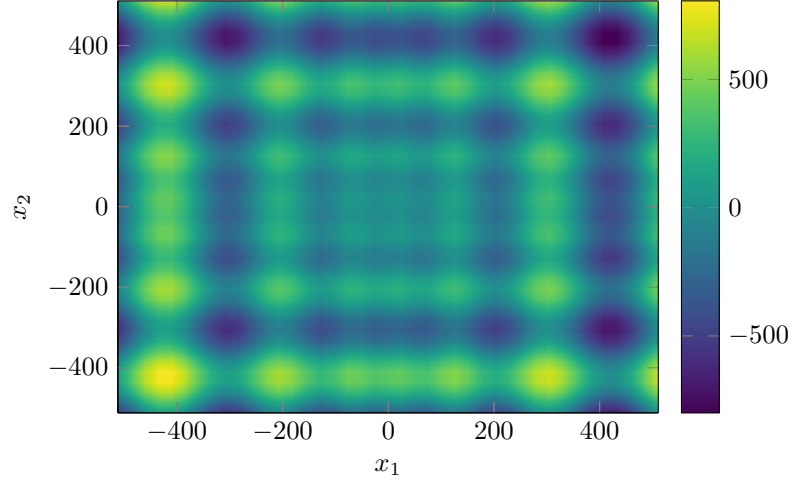


Figure 18.6: Two-variable Rastrigin, $f_6(x_1, x_2)$.

Figure 18.7: Two-variable Schwefel, $f_7(x_1, x_2)$.

$$f_8(\mathbf{x}) = 1 + \sum_{i=1}^n \frac{x_i^2}{4000} - \prod_{i=1}^n \cos\left(\frac{x_i}{\sqrt{i}}\right), \quad x_i \in [-5.12, 5.11]. \quad (18.8)$$

The global minimum is 0 at the origin.

18.1.9 The f9 Function

F_9 is the sine-envelope sine wave, also called `sine-envelope-sine-wave`.

$$f_9(x_1, x_2) = 0.5 + \frac{\sin^2 \sqrt{x_1^2 + x_2^2} - 0.5}{(1 + 0.001(x_1^2 + x_2^2))^2}, \quad x_i \in [-100, 100]. \quad (18.9)$$

The global minimum is 0 at the origin.

18.1.10 The f10 Function

F_{10} is the stretched V sine wave, also called `stretched-v-sine-wave`. Let $r^2 = x_1^2 + x_2^2$.

$$f_{10}(x_1, x_2) = (r^2)^{1/4} \left(1 + \sin^2(50(r^2)^{1/10}) \right), \quad x_i \in [-100, 100]. \quad (18.10)$$

The global minimum is 0 at the origin.

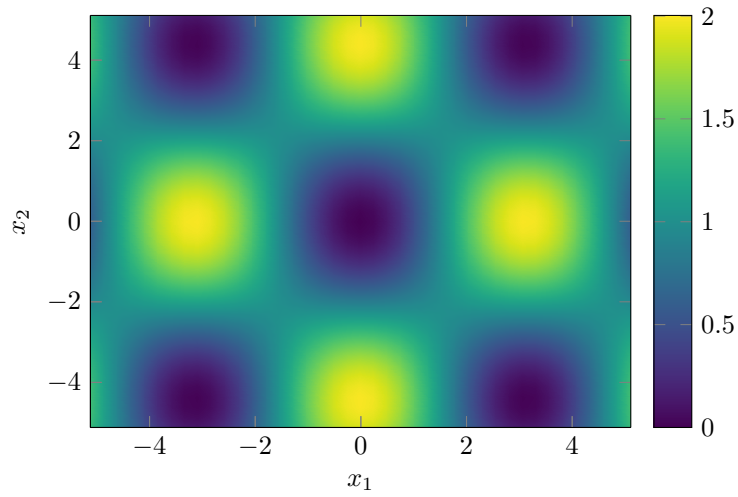


Figure 18.8: Two-variable Griewangk, $f_8(x_1, x_2)$.

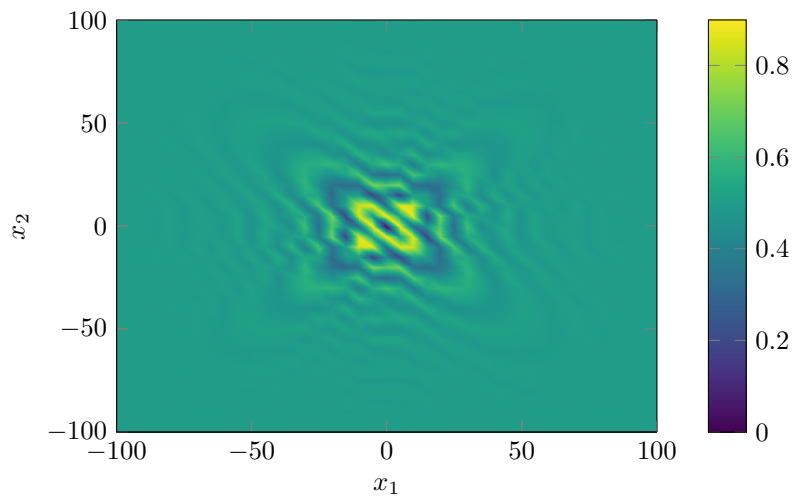


Figure 18.9: f_9 , the sine-envelope sine wave.

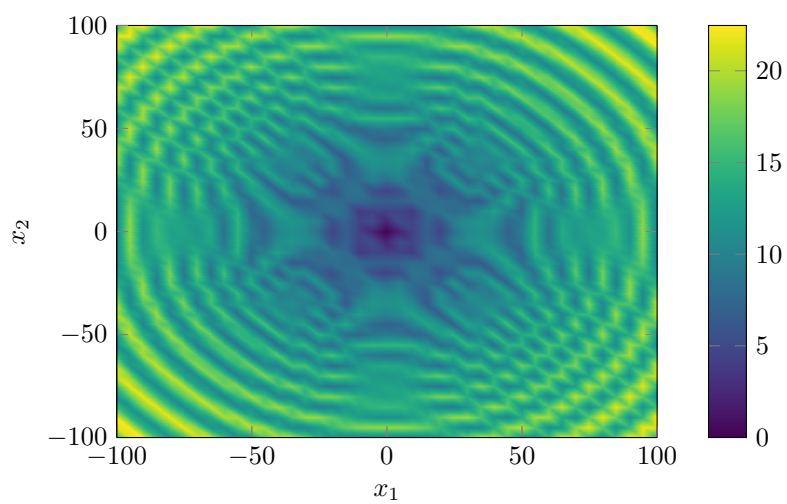


Figure 18.10: f_{10} , the stretched V sine wave.

Bibliography

- [1] BUTZ, M. V., AND WILSON, S. W. An algorithmic description of XCS. In *Advances in Learning Classifier Systems* (2001), vol. 1996 of *Lecture Notes in Artificial Intelligence*, Springer, pp. 253–272.
- [2] DE JONG, K. A. *An Analysis of the Behavior of a Class of Genetic Adaptive Systems*. PhD thesis, University of Michigan, 1975.
- [3] GORE, C. M. A time series classifier. Master’s thesis, Missouri University of Science and Technology, 2008. <https://cgore.com/cgore-thesis-final.pdf>.
- [4] GORE, C. M. Σ : *A Library for ANSI Common Lisp*, 2026. <https://github.com/cgore/sigma>.
- [5] LEFÈVRE, E. *Reminiscences of a Stock Operator*. George H. Doran, New York, 1923. A fictionalized account of Jesse Lauriston Livermore.
- [6] WHITLEY, D., MATHIAS, K., RANA, S., AND DZUBERA, J. Evaluating evolutionary algorithms. *Artificial Intelligence* 85 (1996), 245–276.
- [7] WILSON, S. W. Classifier fitness based on accuracy. *Evolutionary Computation* 3, 2 (1995), 149–175.
- [8] WILSON, S. W. Get real! XCS with continuous-valued inputs. In *Learning Classifier Systems: From Foundations to Applications* (2000), vol. 1813 of *Lecture Notes in Artificial Intelligence*, Springer, pp. 209–219.